

A Formal Language toward the Unification of Model Checking and Performance Evaluation

Andrew S. Miner and Yaping Jing

Department of Computer Science
Iowa State University
{yaping, asminer}@iastate.edu

Abstract. There are currently a variety of logics used to describe properties of discrete-state systems, such as linear temporal logic (LTL) and computation tree logic (CTL), and of discrete-state stochastic systems, such as probabilistic CTL (PCTL) and continuous stochastic logic (CSL). While powerful, these logics are by definition limited to producing only true or false responses. In this work, we introduce a query language, CTML, that operates on real-valued quantities to reason over probabilistic systems. CTML is inspired by, and evolves from stochastic logic. It extends until and weak until with arithmetic operations on real values, while still preserving their conventional semantics of temporal logic. We derive mathematical algorithms for computing each of the operators, show how any PCTL property can be expressed in CTML, and illustrate the expressiveness of CTML with some example performance-related queries.

Keywords: Stochastic Logics, Markov chains

1 Introduction

Over the years, performance modeling has gained acceptance for design and analysis of complex computer and communication systems. This is due in part to the availability of computer software tools to automatically compute desired performance measures. Under such a framework, the system model must be described using an appropriate high-level formalism, and similarly the desired performance measures of the system model must be described formally. The types of performance questions that can be answered ultimately depend on the mechanism by which the performance measures are specified.

For properties that depend only on which states can be reached, and not on the time or probability to reach them, existing temporal logics such as linear temporal logic (LTL) and computation tree logic (CTL) [11] are powerful, widely-used logics for model checking, and can express properties such as “the system never reaches a deadlocked state”. A model checker will then verify if the property is true or false. To express performance-related properties, these logics have been extended by adding or modifying operators (e.g., PLTL [13], PCTL [16], pCTL* [5], DCTL[3], CSL [8], asCSL [6], CSL^{TA} [15]), and can express

properties such as “with probability at least 0.95, the system will not reach a deadlocked state before time 100”. While these logics are powerful, they are limited to producing only true or false responses, precisely because they are *logics* (although in practice, a real-valued response can sometimes be obtained for the outer-most path quantifier).

The classic mechanism for describing performance measures within a high-level model is to use a reward function, that assigns a real value to each underlying state of the model (see for instance [24] for Petri nets or [10] for process algebra). Performance queries can then be expressed in terms of the expected reward at a fixed time (including infinite time for steady-state), or the expected accumulated reward for a time interval. This idea was later extended to capture path information [25], by utilizing a path automata that is combined with the underlying Markov chain via a synchronous product. A limitation of this work is the lack of a formal language for describing queries or a mechanism for constructing the path automata.

To address these limitations, in this work we propose a new, formal language to express performance queries, while retaining the ability to express temporal properties. This performance query language uses operators similar to the familiar temporal operators in logics such as CTL and PCTL, but allows real-valued state and path formulas, rather than booleans. As such, the language is no longer a “logic”, and strictly speaking, the results are not obtained by “model checking” but by performance evaluation. The real-valued nature of the language is natural for expressing performance questions such as, “what is the probability that the system will reach a deadlock state before time t ?”, or “when a message is sent, what is the expected time until it is received?”.

The main contributions of this work can be summarized as follows. First, we define a general framework for measures based on path formulas in Markov chains; this serves as a foundation for the evaluation of queries in the language. The main contribution is the definition of *Computation Tree Measurement Language* (CTML) and the mathematical derivations of efficient algorithms for evaluation of CTML formulas. Additionally, we show that CTML is strictly more expressive than PCTL.

The remainder of the paper is organized as follows. Section 2 presents foundations for path-based CTML measures. Section 3 presents the syntax and semantics of the proposed language. Section 4 presents algorithms for computing the core CTML operators. Section 5 presents query examples for the proposed CTML specification. Section 6 presents related work. Section 7 concludes the paper and gives some directions for future work.

2 Foundations

In this section, we recall some well-known definitions, introduce notation, and define the underlying structures necessary for this work.

We define a (time-homogeneous) discrete-time Markov chain (DTMC) as a triple $(\mathcal{S}, \mathbf{P}, \pi_0)$, where $\mathcal{S}$ is the finite state space of the Markov chain, $\mathbf{P}$ is a

$|\mathcal{S}| \times |\mathcal{S}|$ stochastic matrix with $\mathbf{P}(i, j)$ denoting the probability that the Markov chain is in state j at time $t + 1$, given that it is in state i at time t , and $\boldsymbol{\pi}_0$ is a vector with $\boldsymbol{\pi}_0(i)$ denoting the probability that the Markov chain is in state i at time 0. It is well-known that $\mathbf{P}^n(i, j)$ gives the probability that the Markov chain is in state j at time $t + n$, given that it is in state i at time t .

We say that it is possible to reach state j from state i in a DTMC, denoted $i \rightsquigarrow j$, if there exists an integer $n \geq 0$ such that $\mathbf{P}^n(i, j) > 0$; otherwise, we write $i \not\rightsquigarrow j$. A state i is called *transient* if there exists a state j such that $i \rightsquigarrow j$ and $j \not\rightsquigarrow i$. A state i is called *absorbing* if $i \rightsquigarrow j$ for every $j \neq i$. We say a DTMC is absorbing if every state is either transient or absorbing. If $\mathcal{S}_z$ denotes the set of transient states, and $\mathcal{S}_a$ denotes the set of absorbing states, the matrix $\mathbf{P}$ can be arranged in a block structure with $\mathbf{P}(\mathcal{S}_z, \mathcal{S}_a) = \mathbf{0}$ and $\mathbf{P}(\mathcal{S}_a, \mathcal{S}_a) = \mathbf{I}$. Furthermore, if rows and columns of $\mathcal{S}_z$ are written before rows and columns of $\mathcal{S}_a$, it can be shown that

$$\lim_{n \rightarrow \infty} \mathbf{P}^n = \begin{bmatrix} \mathbf{0} & \mathbf{N} \mathbf{P}(\mathcal{S}_z, \mathcal{S}_a) \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \quad (1)$$

where $\mathbf{N} = (\mathbf{I} - \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))^{-1}$ is called the fundamental matrix [18]. For more thorough discussion of Markov chains, we refer to [18], [26].

Let $\mathcal{X}$ be any set. A collection $\mathcal{G}$ of subsets of $\mathcal{X}$ is a σ -algebra on $\mathcal{X}$ if the following properties hold:

1. If $G \in \mathcal{G}$, then $G^c \in \mathcal{G}$, where G^c denotes the complement of set G .
2. $\mathcal{X} \in \mathcal{G}$. Note, by the above property, this implies $\emptyset \in \mathcal{G}$.
3. For any sequence $G_0, G_1, \dots$ with $G_i \in \mathcal{G}$, $\bigcup_{i=0}^{\infty} G_i \in \mathcal{G}$.

An important property is that, for any collection $\mathcal{C}$ of subsets of $\mathcal{X}$, there is a unique smallest σ -algebra on $\mathcal{X}$ that includes (i.e., is a superset of) $\mathcal{C}$, called the σ -algebra *generated by* $\mathcal{C}$. Let $\mathcal{X}$ be a set, and $\mathcal{G}$ be a σ -algebra on $\mathcal{X}$. A function $\mu : \mathcal{G} \rightarrow [0, \infty]$ is a *measure* if $\mu(\emptyset) = 0$, and for any sequence $G_0, G_1, \dots$ of disjoint sets in $\mathcal{G}$,

$$\mu \left(\bigcup_{i=0}^{\infty} G_i \right) = \sum_{i=0}^{\infty} \mu(G_i).$$

A more rigorous treatment of measure theory may be found for example in [12].

Let $D = (\mathcal{S}, \mathbf{P}, \boldsymbol{\pi}_0)$ be a discrete-time Markov chain. A *path* in D is defined as an infinite sequence $(\pi_0, \pi_1, \dots) \in \mathcal{S}^\omega$. A *path formula* is a function $\psi : \mathcal{S}^\omega \rightarrow \mathbb{R}^*$, where $\mathbb{R}^*$ denotes the set of non-negative reals. For this work, we wish to determine the expected value of a path formula ψ ; in this section, we define some properties on ψ that will allow us to define a measure for this expected value.

A *prefix* in D is a finite sequence $\mathbf{p} = (\pi_0, \dots, \pi_{n-1}) \in \mathcal{S}^n$, or an infinite sequence $\mathbf{p} = (\pi_0, \pi_1, \dots) \in \mathcal{S}^\omega$, where $|\mathbf{p}| = n \in \mathbb{N} \cup \{\omega\}$ is the length of the sequence. For a given prefix $\mathbf{p} = (\pi_0, \dots, \pi_{n-1})$, define $\mathcal{S}_{\mathbf{p}}^\omega$ as the set of all infinite-length paths that start with prefix $\mathbf{p}$. Note that, if $|\mathbf{p}| = n \in \mathbb{N}$, we have

$$\mathcal{S}_{\mathbf{p}}^\omega = \{\pi_0\} \times \dots \times \{\pi_{n-1}\} \times \mathcal{S}^\omega;$$

otherwise, if $|\mathbf{p}| = \omega$, then we have $\mathcal{S}_{\mathbf{p}}^\omega = \{\mathbf{p}\}$. Note $\mathcal{S}_{()}^\omega = \mathcal{S}^\omega$, where $()$ denotes the zero-length sequence. Let $\mathcal{G}_{\mathcal{S}}$ denote the σ -algebra generated by the set $\{\mathcal{S}_{\mathbf{p}}^\omega : \mathbf{p} \text{ is a prefix}\}$. Since for any prefix $\mathbf{p}$, the set $\mathcal{S}_{\mathbf{p}}^\omega$ is isomorphic to some closed interval $[a, b]$, we have that $\mathcal{G}_{\mathcal{S}}$ is isomorphic to the Borel algebra on reals.

We say a prefix $\mathbf{p}$ *determines* ψ if, for any paths $\mathbf{x}, \mathbf{x}' \in \mathcal{S}_{\mathbf{p}}^\omega$, $\psi(\mathbf{x}) = \psi(\mathbf{x}')$; since all paths must have the same value for ψ in this case, we denote this quantity as $\psi(\mathbf{p})$. Note that any infinite prefix determines ψ . We say a path formula ψ is *finitely measurable* on Markov chain $(\mathcal{S}, \mathbf{P}, \pi_0)$ if, for every path $\mathbf{x} \in \mathcal{S}^\omega$ with $\psi(\mathbf{x}) > 0$, either there exists a finite prefix $\mathbf{p}$ with $\mathbf{x} \in \mathcal{S}_{\mathbf{p}}^\omega$ that determines ψ , or the probability measure for path $\mathbf{x}$ is zero. Finally, for any finitely measurable formula ψ , we define the measure $\mu_\psi : \mathcal{G}_{\mathcal{S}} \rightarrow \mathbb{R}^*$ by

$$\mu_\psi(\mathcal{S}_{\mathbf{p}}^\omega) = \begin{cases} \psi(\mathbf{p}) \prod_{i=1}^{|\mathbf{p}|-1} \mathbf{P}(\pi_{i-1}, \pi_i) & \text{if } \mathbf{p} = (\pi_0, \pi_1, \dots) \text{ determines } \psi \\ \sum_{s \in \mathcal{S}} \mu_\psi(\mathcal{S}_{(\mathbf{p}, s)}^\omega) & \text{otherwise} \end{cases}$$

with $\mu_\psi(\emptyset) = 0$. Note that the finitely measurable property avoids the difficulty of having to observe an infinitely-long path to determine ψ , since any such path $\mathbf{x}$ must have $\mu_\psi(\mathcal{S}_{\mathbf{x}}^\omega) = 0$.

3 Computation Tree Measurement Language

Similar to CTL and PCTL, Computation Tree Measurement Language (CTML) is based on state formulas and path formulas. However, CTML formulas are designed to express a real-valued quantity, rather than merely logical truth. Furthermore, the range of values allowed by CTML formulas depends on the operator. To this end, we define a *state formula* as a function $\phi : \mathcal{S} \rightarrow \mathbb{R}^*$, a *restricted state formula* as a function $\varphi : \mathcal{S} \rightarrow [0, 1]$, a *path formula* as a function $\psi : \mathcal{S}^\omega \rightarrow \mathbb{R}^*$, and a *restricted path formula* as a function $\varrho : \mathcal{S}^\omega \rightarrow [0, 1]$. We have a set AF of atomic state formulas, and a set AR of atomic restricted state formulas. We are now ready to define CTML.

3.1 Syntax of CTML

We use path operators X , U , and V corresponding to “next”, “until”, and “weak until” operators in PCTL. We also allow for “time bounded” versions of until and weak until. However, CTML “until” operators are used to either sum or multiply values until a condition is met; as such, each is marked with an arithmetic operator $\odot \in \{+, \times\}$. We have a single “path quantifier”, M , that indicates the expected value (mean) of a path formula. We choose the quantifier M , rather than the potentially more appropriate E , to avoid confusion with the E operator in CTL. Like PCTL and CTL, in CTML the path operators cannot be arbitrarily nested, but rather must appear in path quantifier, path operator pairs. The syntax of CTML is given by the following grammar:

$$\phi ::= f \mid \varphi \mid \phi \odot \phi \mid M\psi, \quad \varphi ::= one \mid zero \mid r \mid \phi \bowtie \phi \mid \varphi \cdot \varphi \mid M\varrho$$

$$\begin{aligned}\psi &::= X\phi \mid \phi U_{\odot}^{\leq t} \phi \mid \phi V_{\odot}^{\leq t} \phi \mid \phi U_{+} \phi \mid \varphi U_{\times} \phi \mid \varphi V_{\times} \phi \\ \varrho &::= X\varphi \mid \varphi U_{\times}^{\leq t} \varphi \mid \varphi V_{\times}^{\leq t} \varphi \mid \varphi U_{\times} \varphi \mid \varphi V_{\times} \varphi\end{aligned}$$

where $\bowtie \in \{\leq, \geq, <, >\}$, $\odot \in \{+, \times\}$, $r \in AR$, and $f \in AF$.

Finally, we note that a “top-level” formula in CTML is a state formula, similar in style to CTL and PCTL, except that we require some “restricted” formulas. Specifically, the unbounded operators $f V_{\times} g$ and $f U_{\times} g$ require that f is restricted to values no greater than one.

3.2 Semantics of CTML

We now give the formal semantics of the operators appearing in the language. Unlike CTL and PCTL, which define a satisfaction relation, we define the state and path formulas as real-valued functions as follows.

- If $h = f \bowtie g$, then $h(s) = 1$ if $f(s) \bowtie g(s)$ holds, otherwise $h(s) = 0$.
- If $h = f \odot g$, then $h(s) = f(s) \odot g(s)$.
- If $h = M\psi$, then $h(s) = \mu_{\psi}(\mathcal{S}_s^{\omega})$. Note that this requires ψ to be finitely measurable on the Markov chain.
- If $\psi = Xf$, then $\psi(\pi_0, \pi_1, \dots) = f(\pi_1)$. Note that formula Xf is determined by any prefix of length 2 or greater, and thus is finitely measurable.
- If $\psi = f U_{\odot}^{\leq t} g$, then for $\mathbf{p} = (\pi_0, \pi_1, \dots)$,

$$\psi(\mathbf{p}) = \begin{cases} \left(\odot_{i=0}^{j-1} f(\pi_i) \right) \cdot g(\pi_j) & \text{if } j \leq t, g(\pi_j) > 0 \text{ and } \forall i < j, g(\pi_i) = 0 \\ 0 & \text{otherwise} \end{cases}$$

with $f U_{\odot}^{\leq \infty} g \equiv f U_{\odot} g$. To see that this path formula is finitely measurable, consider any path $\mathbf{p} = (\pi_0, \pi_1, \dots)$ with $\psi(\mathbf{p}) > 0$. For this path, there must exist a j satisfying the first condition, namely $j \leq t$, $g(\pi_j) > 0$, and $\forall i < j, g(\pi_i) = 0$. Since formula $f U_{\odot}^{\leq t} g$, is determined by the finite prefix $(\pi_0, \dots, \pi_j)$, it is finitely measurable.

- If $\psi = f V_{\odot}^{\leq t} g$, then for $\mathbf{p} = (\pi_0, \pi_1, \dots)$,

$$\psi(\mathbf{p}) = \begin{cases} \odot_{i=0}^t f(\pi_i) & \text{if } \forall i \leq t, g(\pi_i) = 0 \\ \left(\odot_{i=0}^{j-1} f(\pi_i) \right) \cdot g(\pi_j) & \text{if } j \leq t, g(\pi_j) > 0 \text{ and } \forall i < j, g(\pi_i) = 0 \\ 0 & \text{otherwise} \end{cases}$$

with $f V_{\odot}^{\leq \infty} g \equiv f V_{\odot} g$. Now we must show that this path formula is finitely measurable. For cases where t is an integer, any finite prefix $(\pi_0, \dots, \pi_t)$ determines ψ , and thus ψ is finitely measurable. For infinite t , when f is a restricted state formula and $\odot = \times$, we have that $\psi(\mathbf{p})$ is positive for path $\mathbf{p} = (\pi_0, \pi_1, \dots)$ if and only if the product $f(\pi_0)f(\pi_1) \cdots$ has at most finitely many terms less than one. In a finite DTMC, this can occur either if $g(\pi_j) > 0$ for some j , or if the path contains infinitely many “loops” on states where

f evaluates to one. The only such paths with non-zero probability measure are those that eventually reach a recurrent class, where f evaluates to one for every state in the recurrent class. We therefore have that formula $f V_{\times} g$ is finitely measurable, for restricted state formulas f .

The value of a state formula ϕ on a DTMC $D = (\mathcal{S}, \mathbf{P}, \boldsymbol{\pi}_0)$ is given by $D_{\phi} = \sum_{s \in \mathcal{S}} \phi(s) \boldsymbol{\pi}_0(s)$.

We now demonstrate that any PCTL formula may be expressed in CTML. A PCTL state formula ϕ is represented as a restricted state formula ϕ' in CTML, with $\phi'(s) = 1$ if and only if $s \models \phi$, otherwise $\phi'(s) = 0$. Given PCTL state formulas f, g and their CTML equivalents f', g' , we have that the PCTL state formulas $\neg f$, $f \wedge g$, and $f \vee g$ are equivalent to $f' < one$, $f' \cdot g'$, and $(f' + g') > zero$, respectively, while the PCTL path formula $f U^{\leq t} g$ is equivalent to $f' U_{\times}^{\leq t} g'$. The PCTL path quantification $P_{\bowtie v} \psi$ is equivalent to $M\psi' \bowtie v$, for $\bowtie \in \{>, \geq, <, \leq\}$, where ψ' is the CTML equivalent of the PCTL path formula ψ . Since CTML has additional operators with no PCTL counterparts (for example, U_+), it follows that CTML is strictly more expressive than PCTL.

4 Algorithms

We now present algorithms for computing the value of a CTML formula on a given DTMC. The overall approach is similar to that of CTL and PCTL, namely, a parse tree is constructed for the formula, and each operator is computed using a specific algorithm, assuming the operand state formulas are already known. Since a state formula is a function from DTMC states to reals, each state formula can be represented as a vector of $|\mathcal{S}|$ floating-point values. If state formulas f and g are known and represented as vectors $\mathbf{f}$ and $\mathbf{g}$, it is trivial to determine the vector representation $\mathbf{h}$ for state formulas $h = f \bowtie g$ and $h = f \odot g$. Since the state formula for $h = MX f$ is given by $h(s) = \sum_{s_1 \in \mathcal{S}} f(s_1) \mathbf{P}(s, s_1)$, in this case the vector $\mathbf{h}$ can be determined simply as $\mathbf{h} = \mathbf{P}\mathbf{f}$. The algorithms for determining $h = Mf U_{\odot}^{\leq t} g$ and $h = Mf V_{\odot}^{\leq t} g$ are presented below.

4.1 Computing MU

To simplify the computation of $h = M\psi$ for $\psi = f U_{\odot}^{\leq t} g$, we use a modified transition probability matrix $\mathbf{P}'$, in which

1. states s_g for which $g(s_g) > 0$ are made absorbing, this set is denoted $\mathcal{S}_g$; and
2. states s_n from which it is impossible to reach a state in $\mathcal{S}_g$ are also made absorbing, this set is denoted $\mathcal{S}_n$. Note that the formula evaluates to zero for these states.

Note that the remaining states, $\mathcal{S}_z = \mathcal{S} \setminus (\mathcal{S}_g \cup \mathcal{S}_n)$, are all transient. It can be shown that the state formula $M\psi$ evaluated on $(\mathcal{S}, \mathbf{P}, \boldsymbol{\pi}_0)$ is equivalent to $M\psi$ evaluated on $(\mathcal{S}, \mathbf{P}', \boldsymbol{\pi}_0)$: any path $\mathbf{p}$ with $\psi(\mathbf{p}) > 0$ must contain a state π_j with $g(\pi_j) > 0$, and since the prefix $(\pi_0, \dots, \pi_j)$ determines ψ , all that remains is to

demonstrate that measure $\mu_\psi(\mathcal{S}_\mathbf{p}^\omega)$ gives the same value on $\mathbf{P}'$ as it does on $\mathbf{P}$, which follows from the fact that rows of $\mathbf{P}'$ and $\mathbf{P}$ are equal for states in $\mathcal{S}_t$. The benefit of this modification is that, under $\mathbf{P}'$, for any path $\mathbf{p} = (\pi_0, \pi_1, \dots)$ such that $\psi(\mathbf{p}) > 0$, it must be the case that $\pi_t \in \mathcal{S}_g$.

Until with addition For simplicity, we assume that $\mathbf{P}$ has already been modified for U as discussed above. Additionally, without loss of generality, we modify f such that $f(s_g) = 0$ for $s_g \in \mathcal{S}_g$. For bounded until, the state formula for $h_t = Mf U_+^{\leq t} g$ is given by

$$h_t(s_0) = \sum_{(s_1, \dots, s_t) \in \mathcal{S}^t} (f(s_0) + \dots + f(s_{t-1})) \prod_{i=1}^t \mathbf{P}(s_{i-1}, s_i) g(s_t) \quad (2)$$

$$= \sum_{(s_1, \dots, s_t) \in \mathcal{S}^t} f(s_0) \prod_{i=1}^t \mathbf{P}(s_{i-1}, s_i) g(s_t) + \sum_{s_1 \in \mathcal{S}} \mathbf{P}(s_0, s_1) h_{t-1}(s_1) \quad (3)$$

where the recurrence terminates with $h_0(s) = 0$ for all s . If we interpret $h_t(s_0)$ as “the amount of reward accumulated according to f , when the DTMC starts in state s_0 and reaches a state s_t with $g(s_t) > 0$ by time t ”, then the above recurrence corresponds to the property that $h_t(s_0)$ is the amount of reward accumulated at time 0, namely $f(s_0)$ times the probability of reaching s_t with $g(s_t) > 0$ by time t , plus the amount of reward accumulated from time 1 to time $t - 1$ after leaving state s_0 .

The recurrence (3) can be written as the matrix equation

$$\mathbf{h}_t = \mathbf{F} \mathbf{P}^t \mathbf{g} + \mathbf{P} \mathbf{h}_{t-1} \quad (4)$$

with terminating condition $\mathbf{h}_0 = \mathbf{g}$, where matrix $\mathbf{F}$ contains values of state formula f on the diagonal and zeroes elsewhere. For finite t , (4) can be computed in practice using auxiliary vector $\mathbf{x}_t = \mathbf{P}^t \mathbf{g} = \mathbf{P} \mathbf{x}_{t-1}$ with $\mathbf{x}_0 = \mathbf{g}$ to avoid computation of matrix $\mathbf{P}^t$.

For unbounded until, the state formula for $h = Mf U_+ g$ is given by $h = \lim_{t \rightarrow \infty} Mf U_+^{\leq t} g$. Defining $\mathbf{h} = \lim_{t \rightarrow \infty} \mathbf{h}_t$, from (4) we obtain the linear system $(\mathbf{I} - \mathbf{P}) \mathbf{h} = \mathbf{F} (\lim_{t \rightarrow \infty} \mathbf{P}^t) \mathbf{g}$. Since $\mathbf{P}$ is an absorbing DTMC, $\lim_{t \rightarrow \infty} \mathbf{P}^t$ is given by (1). Now, let $\mathbf{y} = \lim_{t \rightarrow \infty} \mathbf{P}^t \mathbf{g}$. Note that $\mathbf{y}[i]$ is equal to the probability that the DTMC eventually reaches a state in $\mathcal{S}_g$, starting from state i . We can split the system based on $\mathcal{S} = \mathcal{S}_z \cup \mathcal{S}_n \cup \mathcal{S}_g$ to obtain

$$\begin{bmatrix} \mathbf{y}(\mathcal{S}_z) \\ \mathbf{y}(\mathcal{S}_n) \\ \mathbf{y}(\mathcal{S}_g) \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \mathbf{N} \mathbf{P}(\mathcal{S}_z, \mathcal{S}_n) & \mathbf{N} \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \\ \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{g}(\mathcal{S}_g) \end{bmatrix}$$

and the solution of this system gives $\mathbf{y}(\mathcal{S}_z) = \mathbf{N} \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \mathbf{g}(\mathcal{S}_g)$, $\mathbf{y}(\mathcal{S}_n) = \mathbf{0}$, and $\mathbf{y}(\mathcal{S}_g) = \mathbf{g}(\mathcal{S}_g)$. Vector $\mathbf{y}(\mathcal{S}_z)$ can be obtained by solving the linear system

$$(\mathbf{I} - \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z)) \mathbf{y}(\mathcal{S}_z) = \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \mathbf{g}(\mathcal{S}_g). \quad (5)$$

Since $\mathbf{h}[s]$ is zero for $s \in \mathcal{S}_n \cup \mathcal{S}_g$, the linear system for $\mathbf{h}$ becomes

$$\begin{bmatrix} \mathbf{I} - \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z) & \mathbf{P}(\mathcal{S}_z, \mathcal{S}_n) & \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{h}(\mathcal{S}_z) \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{F}(\mathcal{S}_z, \mathcal{S}_z) & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{F}(\mathcal{S}_n, \mathcal{S}_n) & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{y}(\mathcal{S}_z) \\ \mathbf{0} \\ \mathbf{g}(\mathcal{S}_g) \end{bmatrix}$$

which produces a single equation

$$(\mathbf{I} - \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))\mathbf{h}(\mathcal{S}_z) = \mathbf{F}(\mathcal{S}_z, \mathcal{S}_z)\mathbf{y}(\mathcal{S}_z). \quad (6)$$

Therefore, we can obtain $\mathbf{h}$ by first solving (5) for $\mathbf{y}$, and then solving (6).

Until with multiplication Again, assume that $\mathbf{P}$ has already been modified for U as discussed above, except we can also add states with $f(s) = 0$ and $g(s) = 0$ to the set $\mathcal{S}_n$. Without loss of generality, we modify f such that $f(s_g) = 1$ for $s_g \in \mathcal{S}_g$. For bounded until, the state formula for $h_t = Mf U_{\times}^{\leq t} g$ is given by

$$h_t(s_0) = \sum_{(s_1, \dots, s_t) \in \mathcal{S}^t} f(s_0) \cdots f(s_{t-1}) \prod_{i=1}^t \mathbf{P}(s_{i-1}, s_i) g(s_t) \quad (7)$$

$$= \sum_{(s_1, \dots, s_t) \in \mathcal{S}^t} f(s_0) \mathbf{P}(s_0, s_1) h_{t-1}(s_1) \quad (8)$$

where the recurrence terminates with $h_0(s) = g(s)$. Recurrence (8) can be written as the matrix equation

$$\mathbf{h}_t = \mathbf{F} \mathbf{P} \mathbf{h}_{t-1} \quad (9)$$

with terminating condition $\mathbf{h}_0 = \mathbf{g}$. For a finite t , $\mathbf{h}_t$ can be computed iteratively. Note that unrolling the recurrence gives us $\mathbf{h}_t = (\mathbf{F} \mathbf{P})^t \mathbf{g}$.

For unbounded until, we obtain $\mathbf{h} = \lim_{t \rightarrow \infty} (\mathbf{F} \mathbf{P})^t \mathbf{g}$. If no value of f is greater than one, then it can be shown that $\lim_{n \rightarrow \infty} (\mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))^n = \mathbf{0}$, where $\mathbf{F}_z$ is shorthand for $\mathbf{F}(\mathcal{S}_z, \mathcal{S}_z)$, since $\mathbf{P}$ is the transition probability matrix of an absorbing DTMC with transient states $\mathcal{S}_z$. It then follows that

$$(\mathbf{I} - \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z)) \sum_{i=0}^{\infty} (\mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))^i = \mathbf{I}$$

and therefore $(\mathbf{I} - \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))^{-1}$ exists. Looking at the solution vector $\mathbf{h}$, if we again split the system based on $\mathcal{S} = \mathcal{S}_z \cup \mathcal{S}_n \cup \mathcal{S}_g$, we can obtain

$$\begin{aligned} \mathbf{h}[\mathcal{S}_z] &= \lim_{n \rightarrow \infty} \left(\sum_{i=0}^n (\mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))^i \right) \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \mathbf{g}(\mathcal{S}_g) \\ &= (\mathbf{I} - \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z))^{-1} \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \mathbf{g}(\mathcal{S}_g), \end{aligned}$$

while $\mathbf{h}[\mathcal{S}_n] = \mathbf{0}$ and $\mathbf{h}[\mathcal{S}_g] = \mathbf{g}(\mathcal{S}_g)$. This implies that the linear system

$$(\mathbf{I} - \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_z)) \mathbf{h}[\mathcal{S}_z] = \mathbf{F}_z \mathbf{P}(\mathcal{S}_z, \mathcal{S}_g) \mathbf{g}(\mathcal{S}_g) \quad (10)$$

can be solved to obtain $\mathbf{h}[\mathcal{S}_z]$.

4.2 Computing MV

Again, for the computation of $h = M\psi$ for $\psi = f V_{\odot}^{\leq t} g$, we use a modified transition probability matrix $\mathbf{P}'$ similar to the case for U : the states s_g for which $g(s_g) > 0$ are made absorbing, and we denote this set of states as $\mathcal{S}_g$. The set $\mathcal{S}_n$ is empty, however, and in this case it is not guaranteed that the remaining states $\mathcal{S} \setminus (\mathcal{S}_g \cup \mathcal{S}_n)$ are all transient. It can be shown that the state formula $M\psi$ evaluated on $(\mathcal{S}, \mathbf{P}, \pi_0)$ is equivalent to $M\psi$ evaluated on $(\mathcal{S}, \mathbf{P}', \pi_0)$.

Weak until with addition We assume that $\mathbf{P}$ has already been modified for V as discussed above, and that $f(s_g) = 0$ for $s_g \in \mathcal{S}_g$. Additionally, after constructing $\mathcal{S}_g$, we modify g such that all zero values are replaced with one. With these modifications, the state formula for $h_t = Mf V_+^{\leq t} g$ is given by

$$h_t(s_0) = \sum_{(s_1, \dots, s_t) \in \mathcal{S}^t} (f(s_0) + \dots + f(s_t)) \prod_{i=1}^t \mathbf{P}(s_{i-1}, s_i) g(s_t) \quad (11)$$

where the recurrence terminates with $h_0(s) = f(s)$ for all s . Since (11) is identical to (2), we obtain the same matrix equation as strong until with addition, $\mathbf{h}_t = \mathbf{F}\mathbf{P}^t \mathbf{g} + \mathbf{P}\mathbf{h}_{t-1}$, except we use a different terminating vector $\mathbf{h}_0$.

CTML does not currently support V_+ in the unbounded case, because such a formula produces values of infinity whenever there is a recurrent class $\mathcal{C}$ with $\mathcal{S}_g \cap \mathcal{C} = \emptyset$, and $f(s) > 0$ for some $s \in \mathcal{C}$. While these cases are easy to determine, issues with infinity in the *language* must be resolved (e.g., determining values for $\infty \geq \infty$ or $\infty \cdot 0$) before CTML can support unbounded V_+ .

Weak until with multiplication Again, assume that $\mathbf{P}$ has already been modified for V as discussed above, except we can add states with $f(s) = 0$ and $g(s) = 0$ to the set $\mathcal{S}_n$. Additionally, we modify f such that $f(s_g) = 1$ for $s_g \in \mathcal{S}_g$, and after these modifications are complete, we modify g so that all zero values are replaced with the value one. Then, the state formula for $h_t = Mf V_{\times}^{\leq t} g$ becomes

$$h_t(s_0) = \sum_{(s_1, \dots, s_t) \in \mathcal{S}^t} f(s_0) \cdots f(s_t) \mathbf{P}(s_0, s_1) \cdots \mathbf{P}(s_{t-1}, s_t) g(s_t) \quad (12)$$

where the recurrence terminates with $h_0(s) = f(s) \cdot g(s)$ for all s . Since (12) is identical to (7), we obtain the same matrix equation as strong until with multiplication, $\mathbf{h}_t = \mathbf{F}\mathbf{P}\mathbf{h}_{t-1}$, but with terminating condition $\mathbf{h}_0 = \mathbf{F}\mathbf{g}$.

For unbounded weak until, consistent with the discussion in Section 3.2, the state formula $h = Mf V_{\times} g$ is equivalent to the state formula $h' = Mf U_{\times} g'$ where $g'(s)$ has value $g(s)$ if $g(s) > 0$, has value one if s is in a recurrent class where f evaluates to one for every state in the recurrent class, and has value zero otherwise. Therefore, an algorithm for $MV_{\times}$ is to determine the recurrent classes (after making states with $g(s) > 0$ absorbing), set the function g' appropriately, and invoke the algorithm for $MU_{\times}$.

4.3 Computational complexity

Assuming we limit our computations to floating-point or rational values, the trivial operations $f \boxtimes g$ and $f \odot g$ can be computed in $O(|\mathcal{S}|)$ time. The operation MXf requires matrix-vector multiplication, which can be done in $O(|D|)$ time, where $|D|$ refers to the size of the DTMC (the number of non-zero entries in matrix $\mathbf{P}$ plus the number of states). The bounded operators $Mf U_{\odot}^{\leq t} g$ and $Mf V_{\odot}^{\leq t} g$ can be computed in $O(t \cdot |D|)$ time, and the unbounded operators $Mf U_{\odot} g$ and $Mf V_{\times} g$ require solution of linear systems, which can be done in polynomial time in $|\mathcal{S}|$, which we denote $O(\text{Poly}(|\mathcal{S}|))$. Therefore, assuming t is at most polynomial in $|\mathcal{S}|$, the value of a CTML formula ϕ can be determined in $O(|\phi| \cdot \text{Poly}(|\mathcal{S}|))$ time, where $|\phi|$ gives the number of operators in formula ϕ . This is similar to the complexity for PCTL [16], but better than the $O(2^{|\phi|} \cdot \text{Poly}(|\mathcal{S}|))$ complexity to compute the probability for a PLTL formula [13].

5 The Dining Philosophers Model

In this section, we show example queries through the classic dining philosophers model [14]. Although performance analysis of the dining philosophers model is quite hypothetical, it serves to illustrate capabilities of CTML. We use a stochastic Petri net model similar to one from [23], where the subnet corresponding to a single philosopher is shown in Figure 1; N copies of this subnet must be connected together in a cycle to model N dining philosophers. In the Petri net, a token in place `idl` signifies a thinking philosopher, while a token in place `fork` signifies an available fork; this is the initial state for all N copies of the subnet. After a while, a philosopher may become hungry (transition `hungry` fires); the philosopher must then obtain a left and right fork (by firing transitions `getl` and `getr`) before eating. Tokens in places `waitl` and `waitr` correspond to waiting for the left and right forks, while tokens in places `hasl` and `hasr` correspond to having obtained the left and right forks. We assume a philosopher immediately begins eating once both forks are obtained. When done eating, a philosopher returns both forks and resumes thinking (transition `release` fires). Note that a philosopher never releases a fork until after eating, and each philosopher may obtain either the left or right fork first, with no regard to availability of other forks. It is well-known that this model has exactly two deadlocked states, corresponding to every philosopher holding their left fork, and to every philosopher holding their right fork. In the following, we describe performance measures through example queries followed by the corresponding CTML specification.

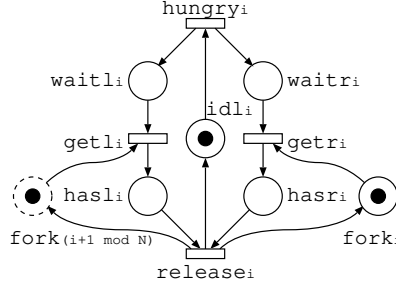


Fig. 1. SPN Model For A Single Dining Philosopher

Query 1: *How much food will philosopher 1 consume, on average, before reaching deadlock?* We define an atomic function *food* that specifies, for each state, how much food philosopher 1 consumes per unit of time (with value 0 when philosopher 1 is not eating); and we define an atomic function *deadlock* with value 1 for the deadlocked states, and 0 otherwise. The query can then be written as

$$M \text{ food } U_+ \text{ deadlock.}$$

Query 2: *How much food will philosopher 1 consume in the first t time units, on average?* Consistent with the above query, we can write this as

$$M \text{ food } V_+^{\leq t} \text{ deadlock,}$$

since no food is consumed in the deadlock states. Alternatively, we can write

$$M \text{ food } V_+^{\leq t} \text{ zero.}$$

Query 3: *What is the probability that philosopher 1 eventually eats?* First, note that for any state formula g , the formula

$$M \text{ one } U_{\times} g$$

computes the sum over all states s with $g(s) > 0$, the probability that s is reached before any other state s' with $g(s') > 0$, multiplied by $g(s)$. We can therefore express the above query as

$$M \text{ one } U_{\times} \text{ eat}$$

where atomic function *eat* returns 1 for any state where philosopher 1 is eating (i.e., has both forks), and is 0 otherwise.

Query 4: *What is the probability that philosopher 1 will obtain the left fork within t time units?* This is similar to the previous query, except we use a time-bounded formula:

$$M \text{ one } U_{\times}^{\leq t} \text{ left,}$$

where *left* returns 1 only for states where philosopher 1 has the left fork.

Query 5: *Given that philosopher 1 obtains the left fork, what is the probability that philosopher 1 eventually eats?* To determine this quantity, we use the fact that $\Pr\{B \mid A\} = \Pr\{B \cap A\} / \Pr\{A\}$. Thus we first determine the probability that philosopher 1 obtains the left fork, and then eats. Since philosopher 1 will keep the left fork until done eating, we can write this as

$$M \text{ one } U_{\times} (M \text{ left } U_{\times} \text{ eat}). \quad (13)$$

To allow philosopher 1 to release the left fork before eating, we would write

$$M \text{ one } U_{\times} ((M \text{ one } U_{\times} \text{ eat}) \cdot \text{left}) \quad (14)$$

instead. Note that the former is preferred when possible, since the linear system for $M \text{ left } U_{\times} \text{ eat}$ has smaller dimension than the one for $M \text{ one } U_{\times} \text{ eat}$. These types of queries (in general) are not expressible with PCTL, even allowing the top-most $P_{=?}$ operation as described in [20]. However, they are expressible in PLTL as $P_{=?}[F(\text{left } U \text{ eat})]$ and $P_{=?}[F(\text{left} \wedge F \text{ eat})]$, respectively. Alternatively, we can note that, for this model, philosopher 1 always obtains the left fork before eating, and write this as

$$M \text{ one } U_{\times} \text{ eat}.$$

Regardless of the formula used, we must divide the result by $M \text{ one } U_{\times} \text{ left}$, which is the probability that philosopher 1 obtains the left fork.

Query 6: *If philosopher 1 just picked up the left fork, then what is the probability that philosopher 1 eats within t time?* This is similar to the previous query, except we must use a time-bounded formula. Since we want time to start counting when the left fork is obtained, we use

$$M \text{ one } U_{\times} (M \text{ left } U_{\times}^{\leq t} \text{ eat}) \quad (15)$$

and divide the result by $M \text{ one } U_{\times} \text{ left}$.

Query 7: *If philosopher 1 just picked up left fork, then what is the expected time until deadlock?* First, we determine the expected time until deadlock starting from each possible state. Then, we filter out all but the states where philosopher 1 has the left fork. We then sum over all left fork states, the probability to reach that one “first” multiplied by the expected time to deadlock starting from that state. This gives us the expression

$$M \text{ one } U_{\times} ((M \text{ one } U_{+} \text{ deadlock}) \cdot \text{left}). \quad (16)$$

Again, we must divide the result by $M \text{ one } U_{\times} \text{ left}$. We believe that this query cannot be expressed in any existing formalism.

We have a prototype implementation, in Java, to illustrate the feasibility of the algorithms presented in Section 4. We generate a CTMC for the dining philosophers model, and study its embedded DTMC, which is fed into our prototype along with the desired CTML queries. In the following, we consider

computation of Query 7 for 10 dining philosophers, which produces a DTMC with 1,860,498 states. The computations were carried out on a computer with a 2.4 Ghz Intel core 2 duo processor and 2GB of 667 MHz RAM, running MacOS X, and the Java virtual machine for JDK version 1.6.

For the “inner query” of *Mone U₊ deadlock*, we have $|\mathcal{S}_g| = 2$ for the two deadlocked states and $|\mathcal{S}_n| = 0$; the query therefore requires solution of two linear systems of dimension $|\mathcal{S}_z| = 1,860,496$. Using Gauss–Seidel and a relative precision of 10^{-6} , this requires roughly 475 seconds of CPU time. For the “outer query”, we have $|\mathcal{S}_g| = 514,229$ states where philosopher 1 has obtained the left fork, $|\mathcal{S}_n| = 1$ from which philosopher 1 will never obtain the left fork (namely, the deadlock state where all philosophers hold the right fork), leaving $|\mathcal{S}_z| = 1,346,268$. This linear system can be solved in about 46 seconds. The overall time required for Query 7 is therefore about 521 seconds.

6 Related Work

There is a body of past work on applying formal methods to the analysis of stochastic systems; much of this (namely, [3, 5, 6, 8, 15, 16]) was discussed in Section 1. Additionally, there is work that computes the *probabilistic reachability* and *expected reachability* for Markov decision processes (MDPs) or its variants. More specifically, [1] handles a probabilistic structure where the duration time is either 0 or 1 between state to state transitions. [22] extended the idea by allowing the duration time to be an arbitrary natural number between state to state transitions. [2] and [21] incorporate the real value between state to state transitions for computing the expected reachability, which is then treated as the stochastic shortest path problem [9] for MDPs.

Approaches most related to this work are the extensions of PCTL with rewards as presented in [4] (which is very similar to [7], except the latter works with CTMCs) and more recently in [19]. Both [4] and [19] present an expected accumulated reward operator that is similar to CTML’s bounded weak until, with addition. However, the CTML until and weak until operators with addition take two operands, whereas the cumulative operator defined in [4] takes one operand ϕ that is accumulated over states that satisfy ϕ , and the cumulative operator in [19] accumulates a reward variable that is given by the reward structure. Except for rewards defined on edges which are allowed in [19] but not in CTML, these cumulative operators can be expressed using CTML’s bounded weak until operator by using *zero* for the right operand and adjusting the time to be $t - 1$. [19] also presents a reachability operator R that is similar to CTML’s unbounded until, with addition, except that R takes a single operand, similar to CTL’s F operator. There are two main differences between R and CTML’s unbounded until operator using *zero* for the left operand: first, R requires that the destination states are eventually reached with probability one, while U does not; second, CTML can use real values to distinguish the destination states, while R cannot. Furthermore, we note that Queries 5–7 cannot be expressed using the extensions presented in [4] or [19].

Another work we would like to mention is *performance trees* [27], as they can describe results of various types (real-valued or otherwise, including distributions). Unlike CTML, however, performance trees are a more general framework or interface that relies on the existing performance evaluation algorithms (such as passage time distributions [17]) as well as some of the existing model checking algorithms for the expression of both logic and real-valued measures.

7 Conclusions and Future Work

We have introduced a formal query language, CTML, for describing quantitative performance measures. The combination of nesting and real-valued state formulas, unique to CTML, extend the expressive power of the familiar temporal operators. CTML can express any query expressible in PCTL, some queries expressible in PLTL and not in PCTL, and some queries that cannot be expressed in any other existing logic (to the best of our knowledge). Our CTML algorithm has complexity similar to that of PCTL, but better than PLTL.

We are currently working on extending the language to handle additional types of queries. For instance, CTML cannot express queries of the form “what is the probability that a given path formula has value larger than x ”. This type of query requires an extension of CTML with appropriate operators, and development of efficient algorithms for those operators. Incorporation of existing operators, such as CSL’s steady-state operator, is another area for investigation.

Another direction for future work is to apply CTML or a similar language for continuous time Markov chains or semi-Markov processes. For unbounded until and unbounded weak until queries, this is fairly straightforward: since CTML can handle real-valued state formulas, we can instead analyze the embedded DTMC using CTML, and scale the state formulas by the expected time spent in each state. The time bounded versions of these formulas are not so straightforward to handle, and will likely require significant changes.

Acknowledgment

This work is supported in part by the National Science Foundation under grant CNS-0546041.

References

1. de Alfaro, L.: Temporal logics for the specification of performance and reliability. In: STACS '97. LNCS, vol. 1200, pp. 165–176. Springer (1997)
2. de Alfaro, L.: Computing minimum and maximum reachability times in probabilistic systems. In: CONCUR '99. LNCS, vol. 1664, pp. 66–81. Springer (1999)
3. de Alfaro, L., Faella, M., Henzinger, T.A., Majumdar, R., Stoelinga, M.: Model checking discounted temporal properties. Theor. Comput. Sci. 345(1), 139–170 (2005)

4. Andova, S., Hermanns, H., Katoen, J.P.: Discrete-time rewards model-checked. In: FORMATS '03. LNCS, vol. 2791, pp. 88–104. Springer (2003)
5. Aziz, A., Singhal, V., Balarin, F.: It usually works: The temporal logic of stochastic systems. In: CAV 1995. LNCS, vol. 939, pp. 155–165. Springer (1995)
6. Baier, C., Cloth, L., Haverkort, B., Kuntz, M., Siegle, M.: Model checking Markov chains with actions and state labels. *IEEE Trans. Softw. Eng.* 33, 209–224 (2007)
7. Baier, C., Haverkort, B., Hermanns, H., Katoen, J.P.: On the logical characterisation of performability properties. In: Automata, Languages and Programming. LNCS, vol. 1853, pp. 780–792 (2000)
8. Baier, C., Haverkort, B., Hermanns, H., Katoen, J.P.: Model-checking algorithms for continuous-time Markov chains. *IEEE Trans. Softw. Eng.* 29(6), 524–541 (2003)
9. Bertsekas, D.P., Tsitsiklis, J.N.: An analysis of stochastic shortest path problems. *Math. Oper. Res.* 16(3), 580–595 (1991)
10. Clark, G., Gilmore, S., Hillston, J.: Specifying performance measures for PEPA. In: Proc. 5th Int. AMAST Workshop on Real-Time and Probabilistic Systems. LNCS, vol. 1601, pp. 211–227. Springer (1999)
11. Clarke, E.M., Grumberg, O., Peled, D.A.: Model Checking. MIT Press (1999)
12. Cohn, D.L.: Measure Theory. Birkhäuser, Boston (1980)
13. Courcoubetis, C., Yannakakis, M.: The complexity of probabilistic verification. *J. ACM* 42(4), 857–907 (1995)
14. Dijkstra, E.: Hierarchical ordering of sequential processes. *Acta Inf.* 1, 115–138 (1971)
15. Donatelli, S., Haddad, S., Sproston, J.: CSL^{TA}: an expressive logic for continuous-time Markov chains. In: QEST '07. pp. 31–40 (2007)
16. Hansson, H., Jonsson, B.: A logic for reasoning about time and reliability. *Formal Aspects of Computing* 6(5), 512–535 (1994)
17. Harrison, P.G., Knottenbelt, W.J.: Passage time distributions in large Markov chains. *SIGMETRICS Perf. Eval. Rev.* 30(1), 77–85 (2002)
18. Kemeny, J., Snell, J.: Finite Markov Chains. D. Van Nostrand, Princeton, NJ (1960)
19. Kwiatkowska, M.: Quantitative verification: models techniques and tools. In: ESEC-FSE '07. pp. 449–458. ACM (2007)
20. Kwiatkowska, M., Norman, G., Parker, D.: PRISM: probabilistic model checking for performance and reliability analysis. *SIGMETRICS Perf. Eval. Rev.* 36(4), 40–45 (2009)
21. Kwiatkowska, M., Norman, G., Parker, D., Sproston, J.: Performance analysis of probabilistic timed automata using digital clocks. In: FORMATS '03. LNCS, vol. 2791, pp. 105–120. Springer (2003)
22. Laroussinie, F., Sproston, J.: Model checking durational probabilistic systems. In: FoSSaCS. LNCS, vol. 3441, pp. 140–154. Springer (2005)
23. Miner, A.S.: Implicit GSPN reachability set generation using decision diagrams. *Perf. Eval.* 56(1–4), 145–165 (Mar 2004)
24. Muppala, J.K., Ciardo, G., Trivedi, K.S.: Modeling using stochastic reward nets. In: MASCOTS '93. pp. 367–372. Society for Computer Simulation (1993)
25. Obal, W.D., Sanders, W.H.: State-space support for path-based reward variables. *Perf. Eval.* 35(3–4), 233–251 (1999)
26. Stewart, W.: Introduction to the Numerical Solution of Markov Chains. Princeton Univ. Press (1994)
27. Suto, T., Bradley, J.T., Knottenbelt, W.J.: Performance trees: Expressiveness and quantitative semantics. In: QEST '07. pp. 41–50. IEEE Computer Society (2007)